编译原理

第一章 绪论

哈尔滨工业大学 陈鄞

本章内容

1.1 什么是编译

1.2 编译系统的结构

1.3 编译程序的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

1.1 什么是编译?

类似于数学定义或
自然语言的简洁形式

高级语言
(High Level Language)

- 接近人类表达习惯
- 不依赖于特定机器
- 编写效率高

引入助记符

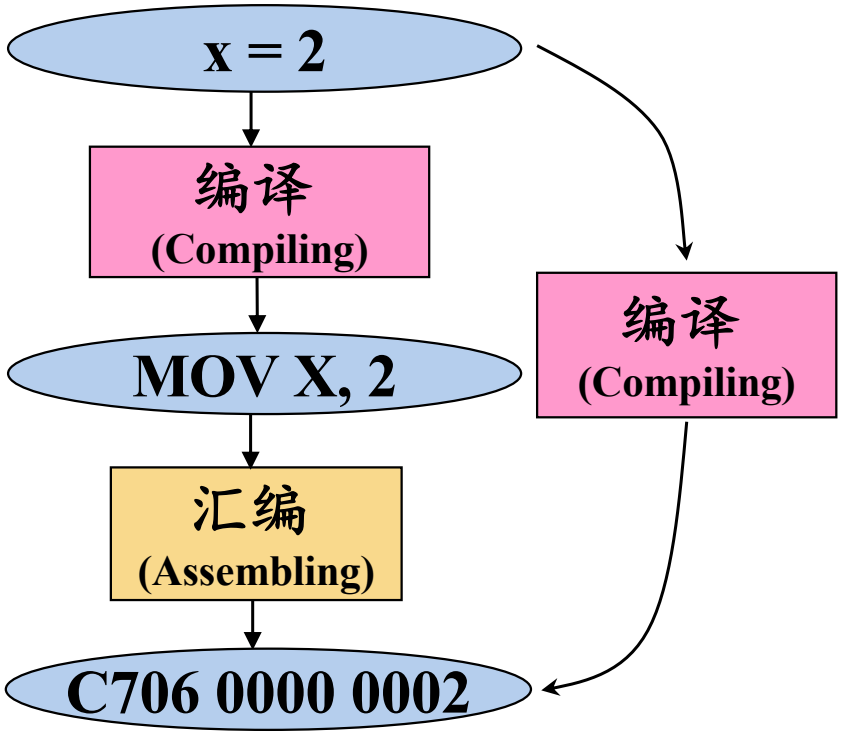
汇编语言
(Assembly Language)

- 依赖于特定机器，非计算机专业人员使用受限制
- 编写效率依然很低

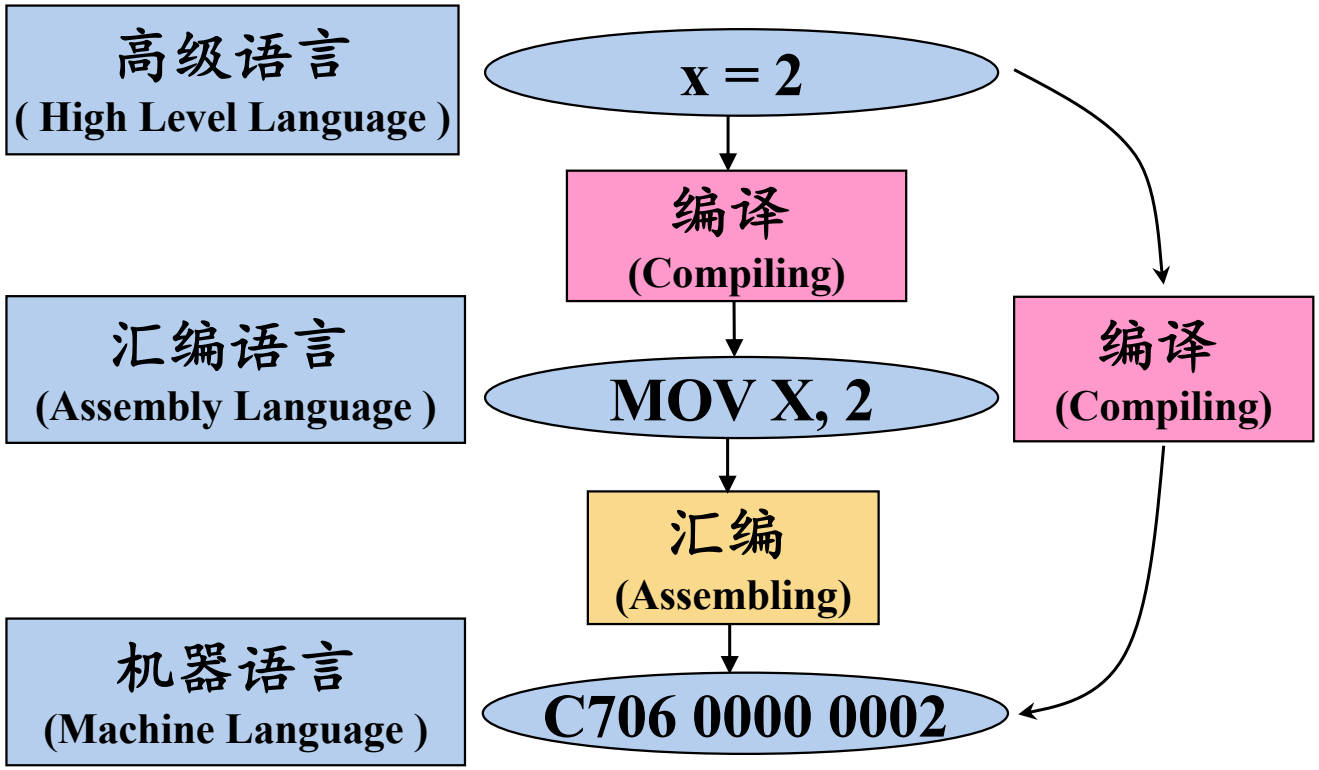
可以被计算机直接理解

机器语言
(Machine Language)

- 与人类表达习惯相去甚远
- 难记忆
- 难编写、难阅读
- 易写错



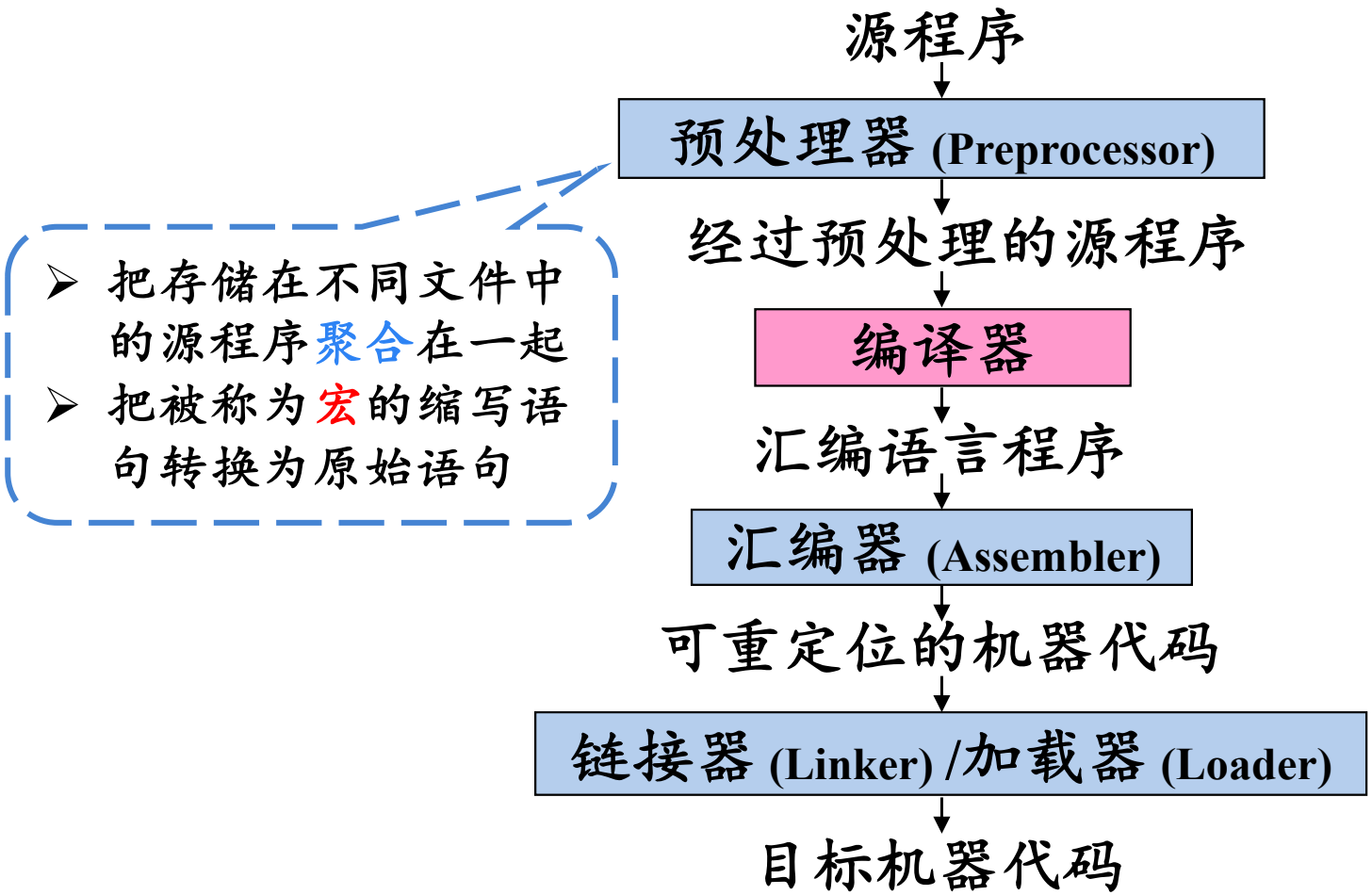
1.1 什么是编译?



编译：将高级语言翻译成汇编语言或机器语言的过程

源语言　　目标语言

编译器在语言处理系统中的位置



编译器在语言处理系统中的位置

源程序

↓

预处理器 (Preprocessor)

↓

经过预处理的源程序

↓

编译器

↓

汇编语言程序

↓

汇编器 (Assembler)

↓

可重定位的机器代码

↓

链接器 (Linker) /加载器 (Loader)

↓

目标机器代码

可重定位(Relocatable):

在内存中存放的起始位置L不是固定的

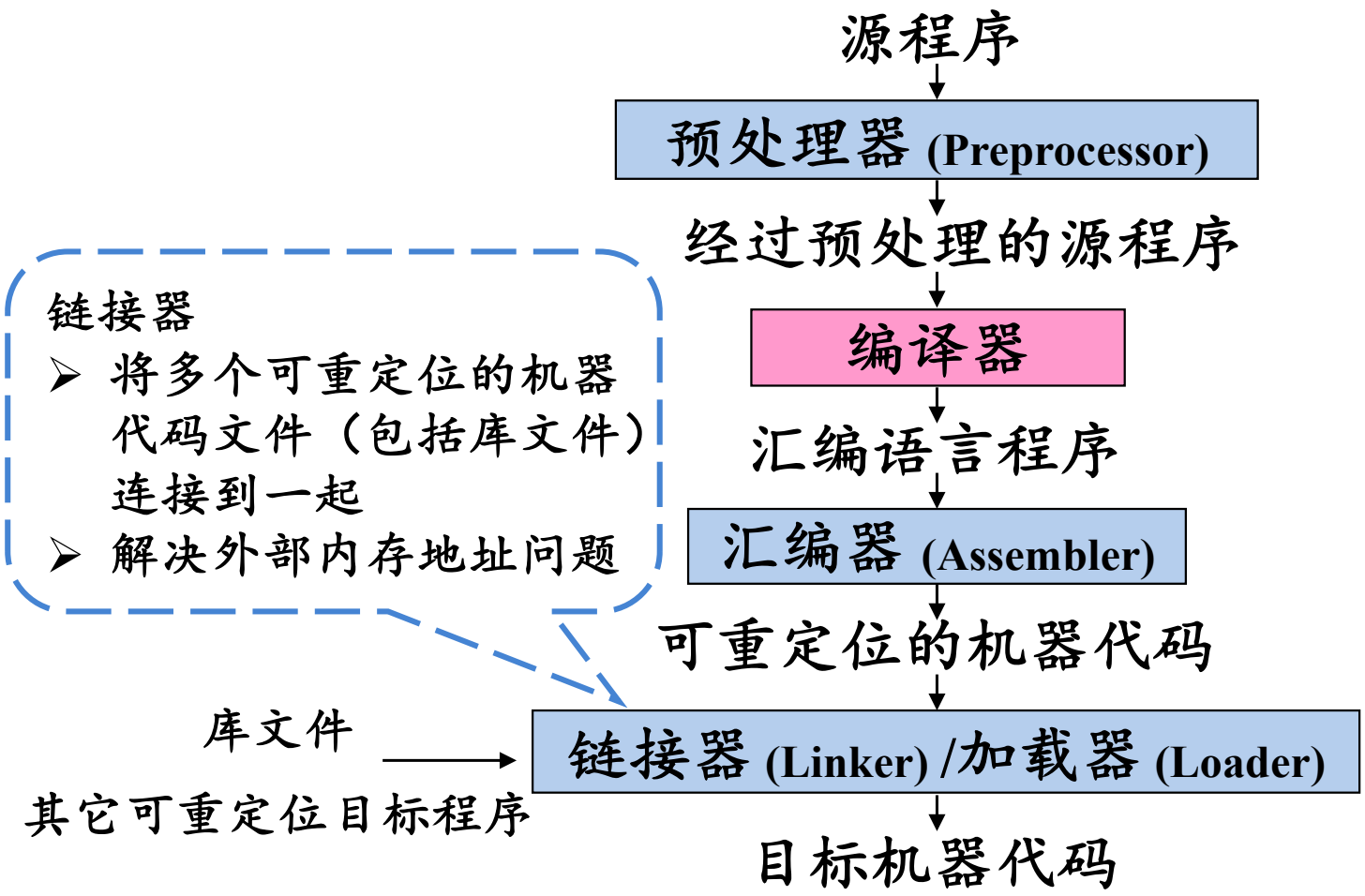
加载器:

修改可重定位地址;

将修改后的指令和数据放到内存中适当的位置

起始位置 + 相对地址 = 绝对地址

编译器在语言处理系统中的位置



提纲

1.1 什么是编译

1.2 编译系统的结构

1.3 编译程序的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

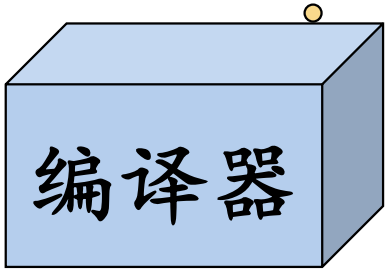
1.2 编译系统的结构

```c
#include<stdio.h>
int main()
{
    int a,b,ge,shi,bai,m,n,i,number;
    printf("请输入(输入完两个数按一次回车键): \n");
    scanf("%d %d",&a,&b);
    while(a!=0,b!=0)
        scanf("%d %d",&a,&b);
    if(a>=100,b<=999)
        if(a>b)
            m=b,n=a;
        else
            m=a,n=b;
        for(i=m;i<=n;i
        {
            bai=i/100;
            shi=(i%100)/10;
            ge=i%10;
            if(i==bai*bai*bai+shi*shi*shi+ge*ge*ge)
            printf("%5d",i);
            number++;
            printf("\n");
        }
        if(number==0)
            printf("no\n");
}
```

```asm
59      call sc
60      mov  al, [si - 1]
61      call sc
62      mov  al, [si]
63      call sc
64      mov  al, ' '
65      call sc
66      ret
67 storechr endp
68 ;清屏，无入口参数
69 clearcrt proc near
70      mov  ax, 0600h
71      mov  bh, 07h
72      mov  cx, 00h
73      mov  dx, 184fh
74      int  10h
                                  no
                                  rd
84        mov  ah, 02h
85        mov  dl, cr
86        int  21h
87        mov  ah, 02h
88        mov  dl, lf
89        int  21h
90      .untilcxz
91      ret
92 nextlien endp
93          end
94
```

机器是如何自动翻译的？

高级语言程序



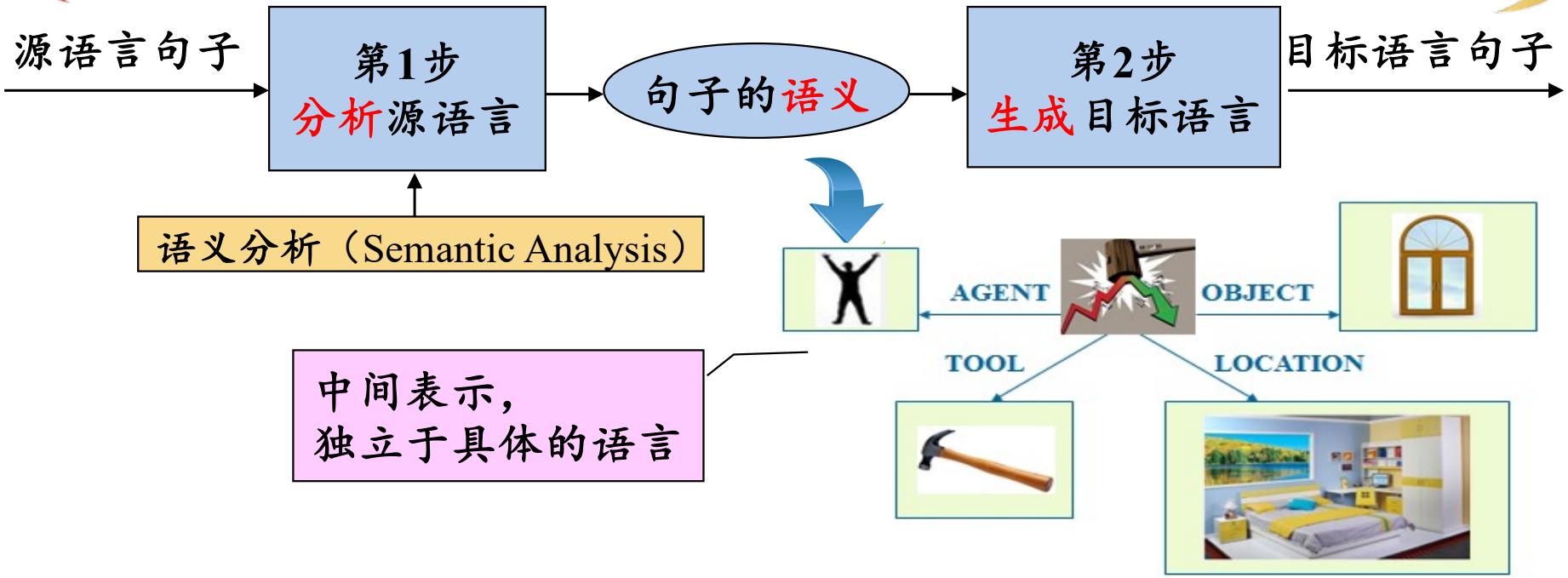
汇编语言程序/机器语言程序

人工英汉翻译的例子

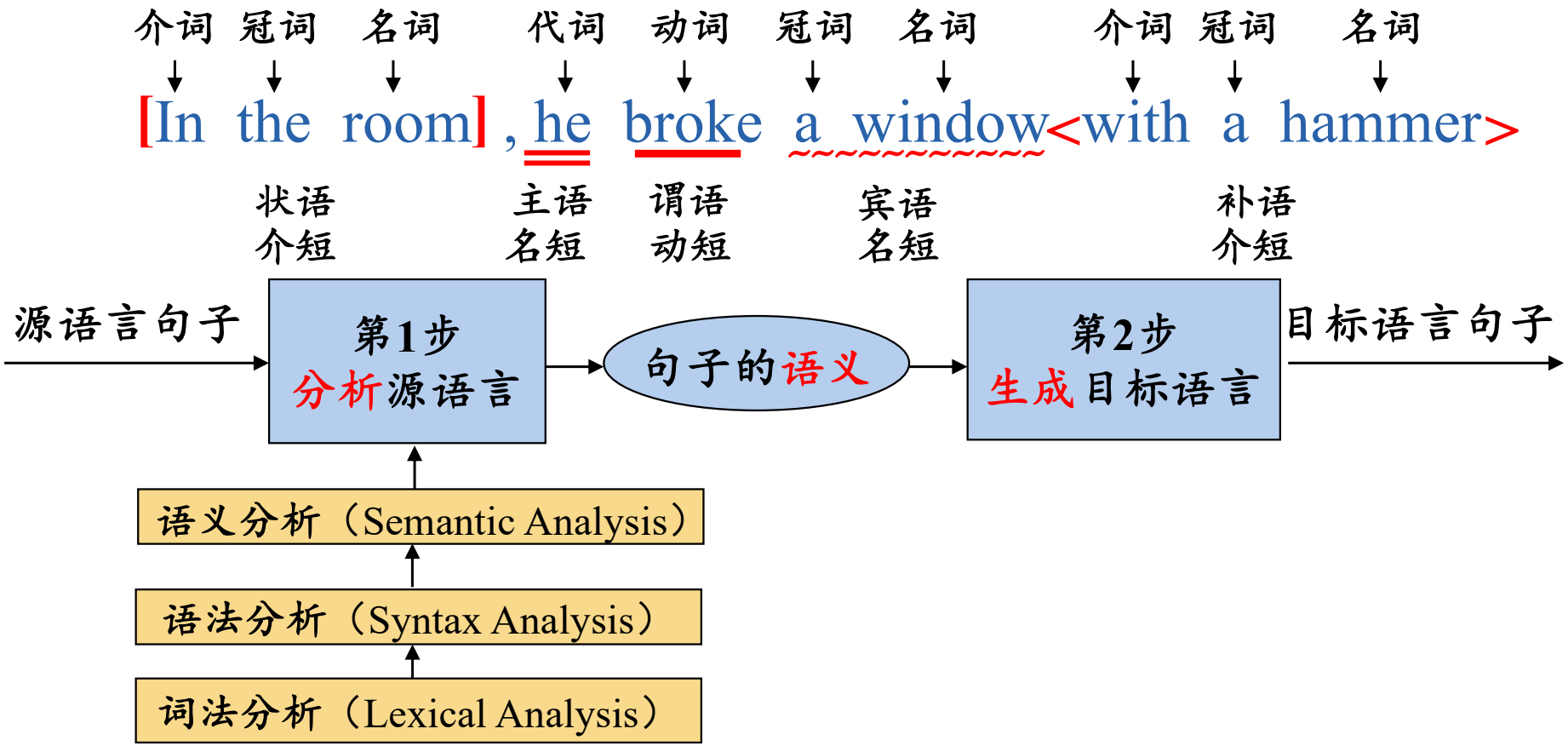
在房间里，他用锤子砸了一扇窗户。

[In the room] , he broke a window<with a hammer>

状语　主语　谓语　宾语　补语



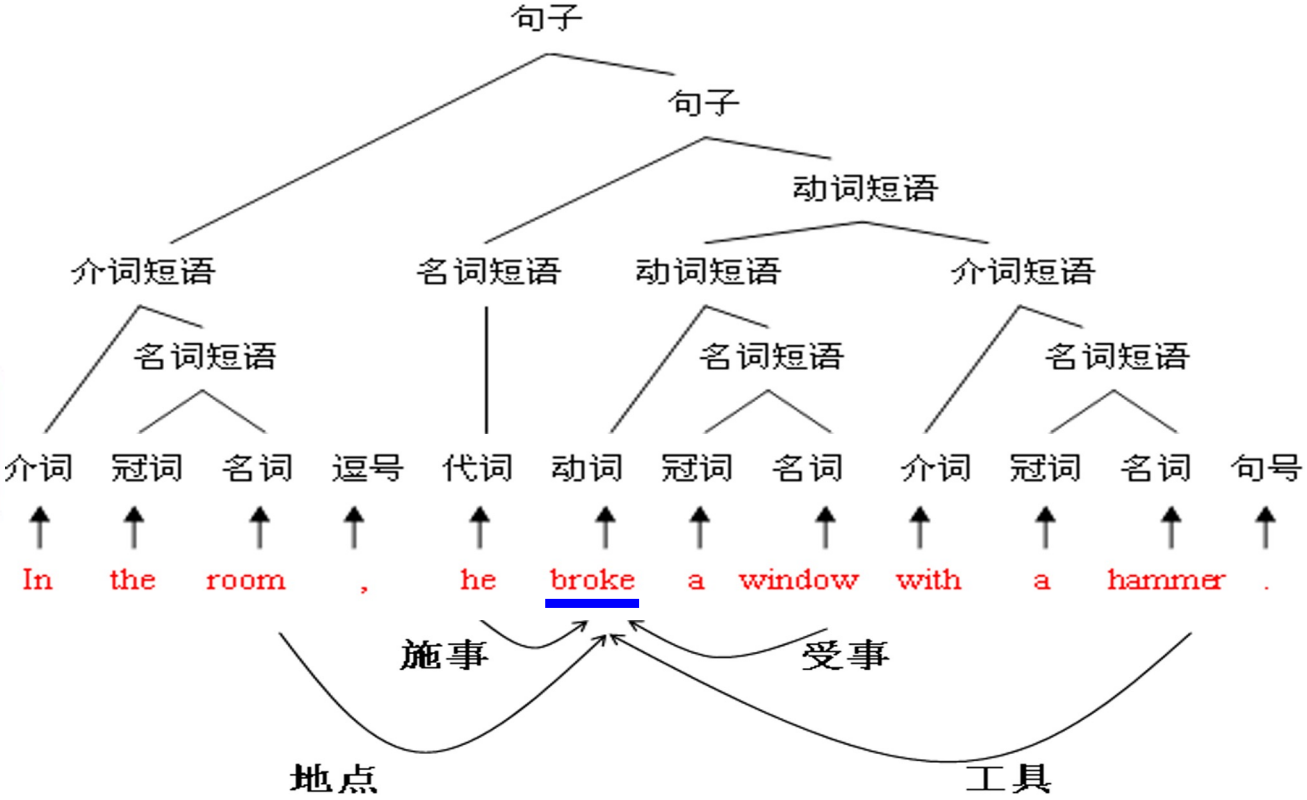
人工英汉翻译的例子


介词 冠词 名词 代词 动词 冠词 名词 介词 冠词 名词
[In the room] , he broke a window<with a hammer>
状语 介短
主语 名短
谓语 动短
宾语 名短
补语 介短
源语言句子
第1步 分析源语言
句子的语义
第2步 生成目标语言
目标语言句子
语义分析（Semantic Analysis）
语法分析（Syntax Analysis）
词法分析（Lexical Analysis）

人工英汉翻译的例子

In the room , he broke a window with a hammer

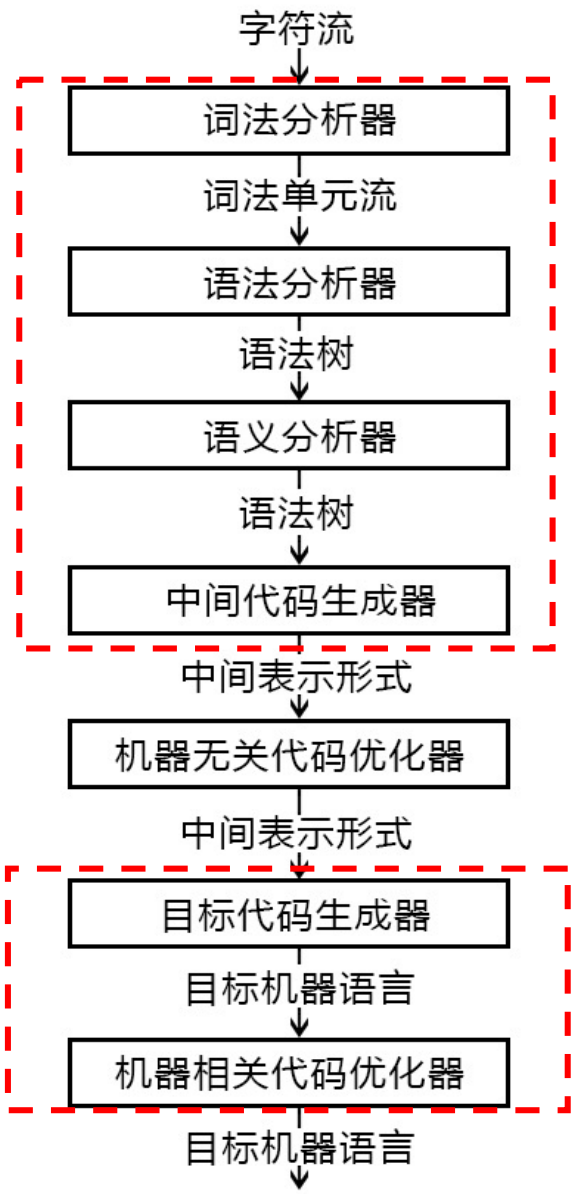
- 词法分析
- 语法分析
- 语义分析



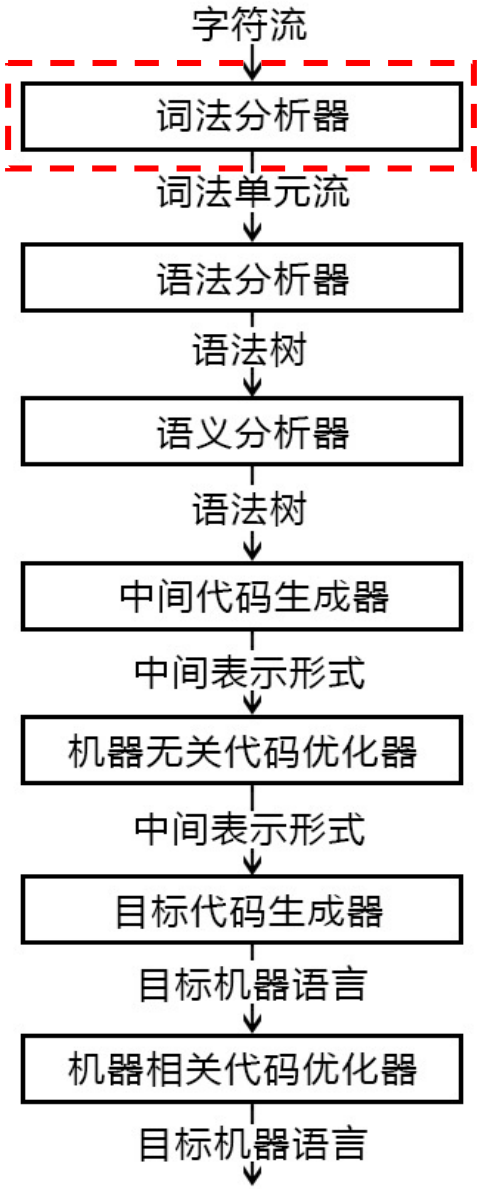
编译器的结构

分析部分/
前端(front end):
与源语言相关

综合部分/
后端(back end):
与目标语言相关



编译器的结构


字符流
词法分析器
词法单元流
语法分析器
语法树
语义分析器
语法树
中间代码生成器
中间表示形式
机器无关代码优化器
中间表示形式
目标代码生成器
目标机器语言
机器相关代码优化器
目标机器语言

词法分析/扫描(*Scanning*)

➢ 词法分析的主要任务

从左向右逐行扫描源程序的字符，识别出各个单词，确定单词的类型。

将识别出的单词转换成统一的机内表示——词法单元(token)形式

token：＜种别码，属性值＞

	单词类型	种别	种别码
1	关键字	program、if、else、then、…	一词一码
2	标识符	变量名、数组名、记录名、过程名、…	多词一码
3	常量	整型、浮点型、字符型、布尔型、…	一型一码
4	运算符	算术（+ - * / ++ --） 关系（> < == != >= <=） 逻辑（& | ~）	一词一码 或 一型一码
5	界限符	; ( ) = { } …	一词一码

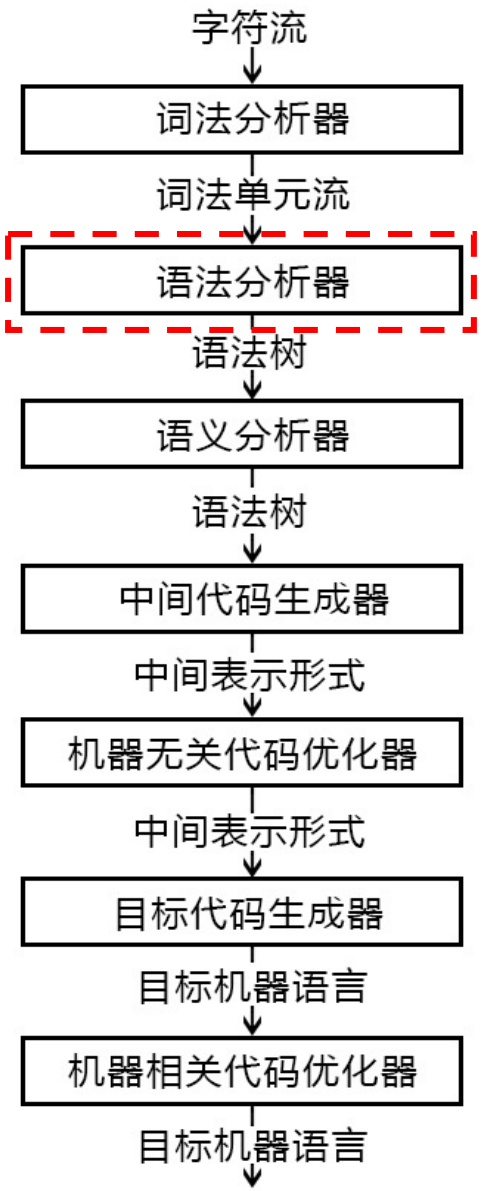
例：词法分析后得到的token序列

- 输入　while(value!=100){num++;}
- 输出

1	while	<	WHILE	,	- >
2	(	<	SLP	,	- >
3	value	<	IDN	,	value >
4	!=	<	NE	,	- >
5	100	<	CONST	,	100 >
6	)	<	SRP	,	- >
7	{	<	LP	,	- >
8	num	<	IDN	,	num >
9	++	<	INC	,	- >
10	;	<	SEMI	,	- >
11	}	<	RP	,	- >

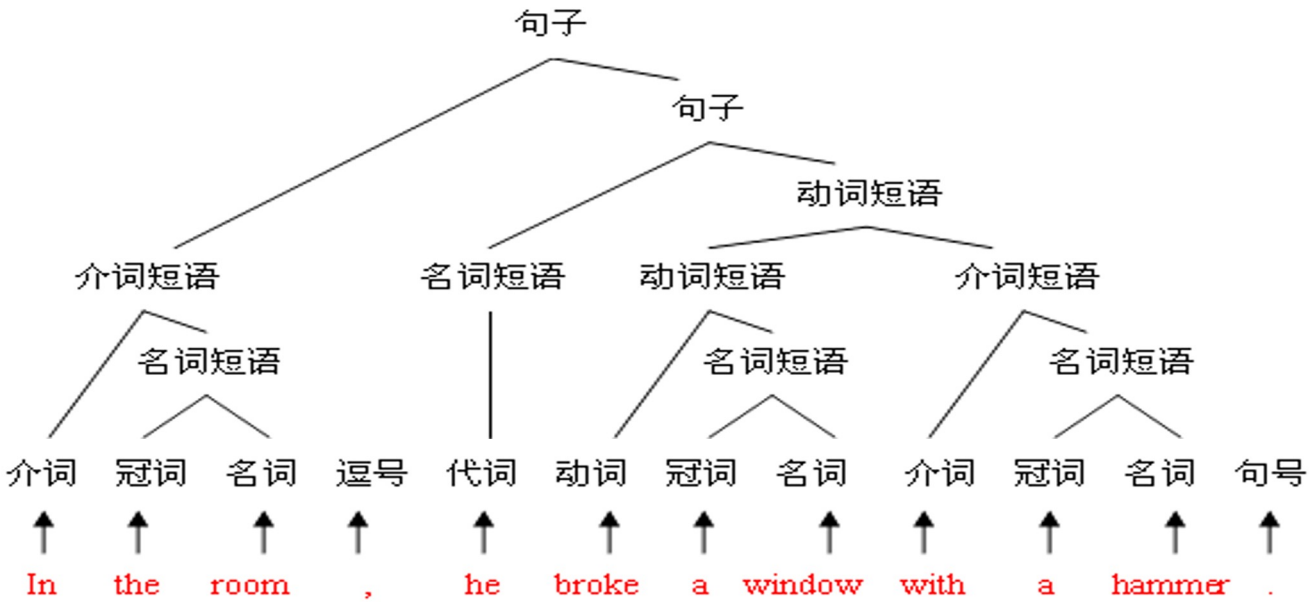
如何实现
词法分析器？

编译器的结构


字符流
词法分析器
词法单元流
语法分析器
语法树
语义分析器
语法树
中间代码生成器
中间表示形式
机器无关代码优化器
中间表示形式
目标代码生成器
目标机器语言
机器相关代码优化器
目标机器语言

语法分析（*parsing*）

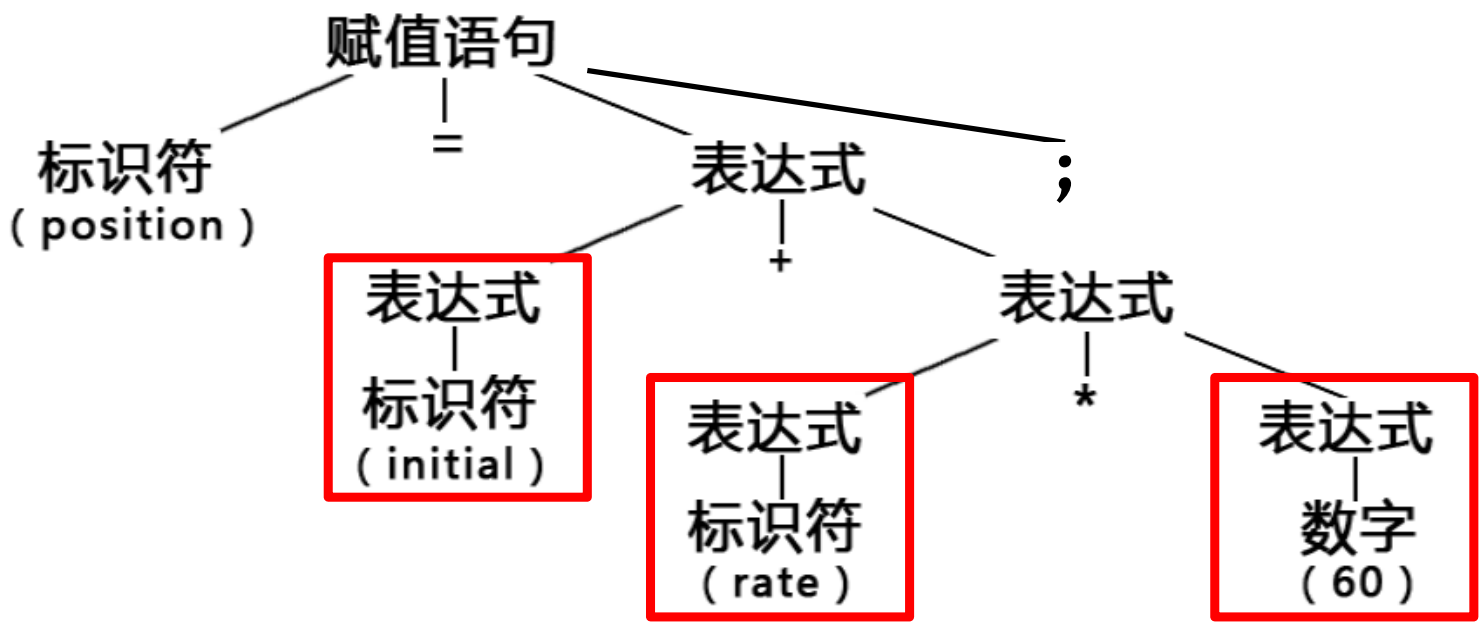
- 语法分析器(parser)从词法分析器输出的token序列中识别出各类短语，并构造语法分析树(parse tree)
  - 语法分析树描述了句子的语法结构



例1：赋值语句的分析树

position = initial + rate * 60 ;

<id, position> <=> <id,initial> <+> <id, rate> <*> <num,60> <;>



例2：变量声明语句的分析树

- 文法：

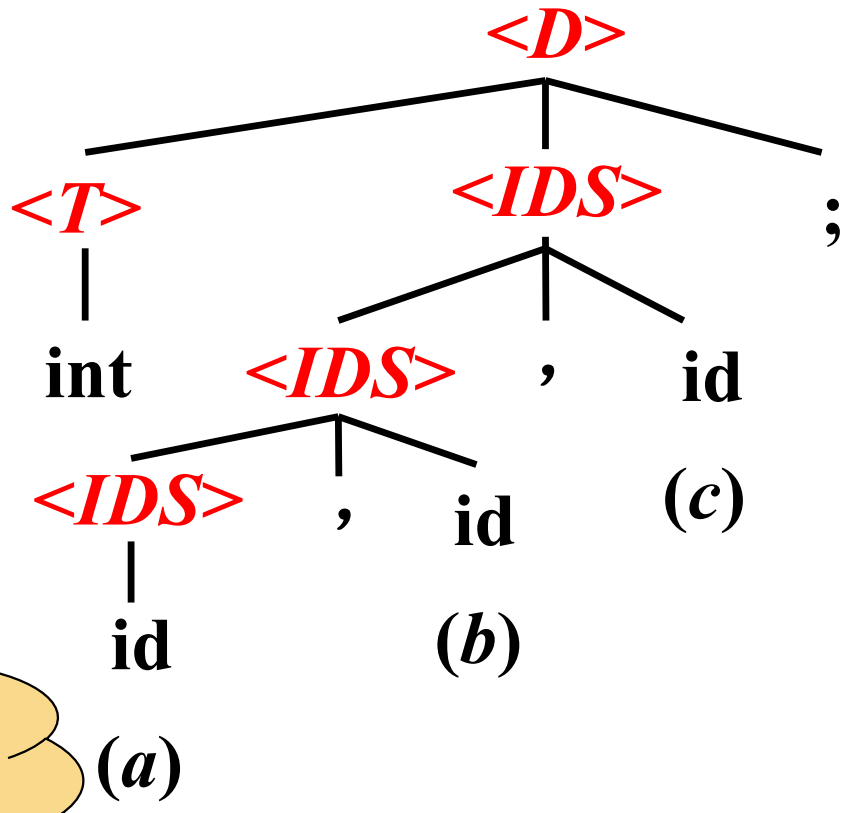
  <*D*> → <*T*> <*IDS*>;

  <*T*> → int | real | char | bool

  <*IDS*> → id | <*IDS*>, id

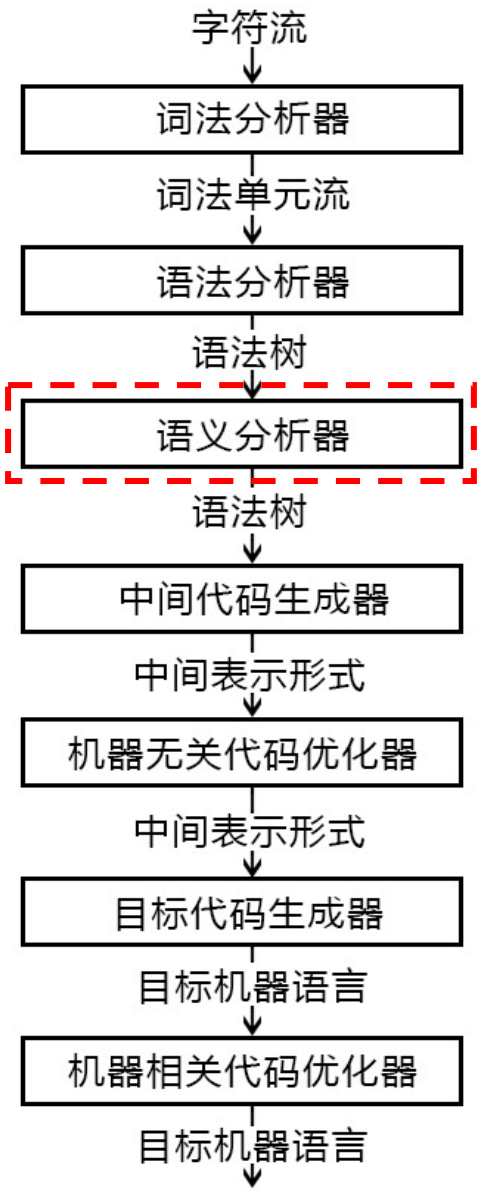
- 输入：

  int *a* , *b* , *c* ;



如何根据语法规则为输入句子构造分析树？

编译器的结构


字符流
词法分析器
词法单元流
语法分析器
语法树
语义分析器
语法树
中间代码生成器
中间表示形式
机器无关代码优化器
中间表示形式
目标代码生成器
目标机器语言
机器相关代码优化器
目标机器语言

语义分析的主要任务

- 收集标识符的属性信息
  - 种属 (Kind)
    - 简单变量、复合变量（数组、记录、…）、过程、…

语义分析的主要任务

- 收集标识符的属性信息
  - 种属 (Kind)
  - 类型 (Type)
    - 整型、实型、字符型、布尔型、指针型、…

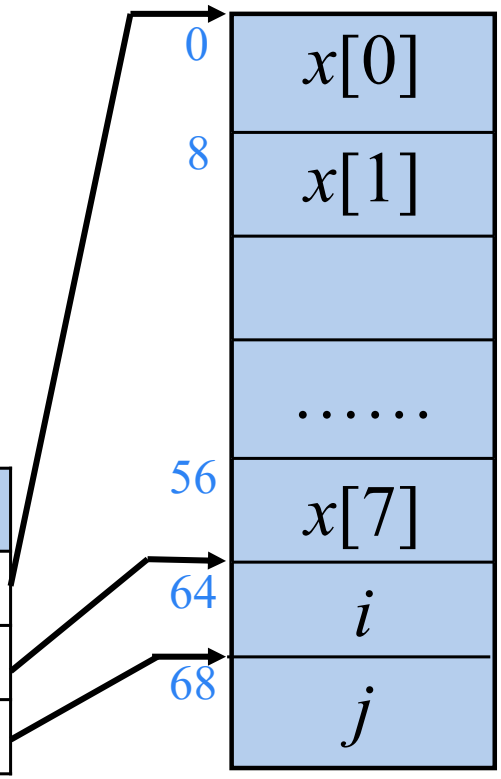
语义分析的主要任务

- 收集标识符的属性信息
  - 种属 (Kind)
  - 类型 (Type)
  - 存储位置、长度

例：

```
begin
    real x[8];
    integer i, j;
    ……
end
```

名字	相对地址
x	0
i	64
j	68



语义分析的主要任务

- 收集标识符的属性信息
  - 种属 (Kind)
  - 类型 (Type)
  - 存储位置、长度
  - 值
  - 作用域
  - 参数和返回值信息
    - 参数个数、参数类型、参数传递方式、返回值类型、…

语义分析的主要任务

- 收集标识符的属性信息
  - 种属 (Kind)
  - 类型 (Type)
  - 存储位置、长度
  - 值
  - 作用域
  - 参数和返回值信息

符号表(Symbol Table)

NAME	TYPE	KIND	VAL	ADDR
SIMPLE	整	简变		
SYMBLE	实	数组		
TABLE	字符	常数		
⋮	⋮	⋮	⋮	⋮

符号表是用于存放标识符的属性信息的数据结构

语义分析的主要任务

符号表中为什么要设计字符串表这样一种数据结构？

- 收集标识符的属性信息
  - 种属 (Kind)
  - 类型 (Type)
  - 存储位置、长度
  - 值
  - 作用域
  - 参数和返回值信息

符号表(Symbol Table)

NAME		TYPE	KIND	VAL	ADDR
	6	整	简变		
	6	实	数组		
	5	字符	常数		
⋮	⋮	⋮	⋮	⋮	⋮

字符串表

S	I	M	P	L	E	S	Y	M	B	L	E	T	A	B	L	E		

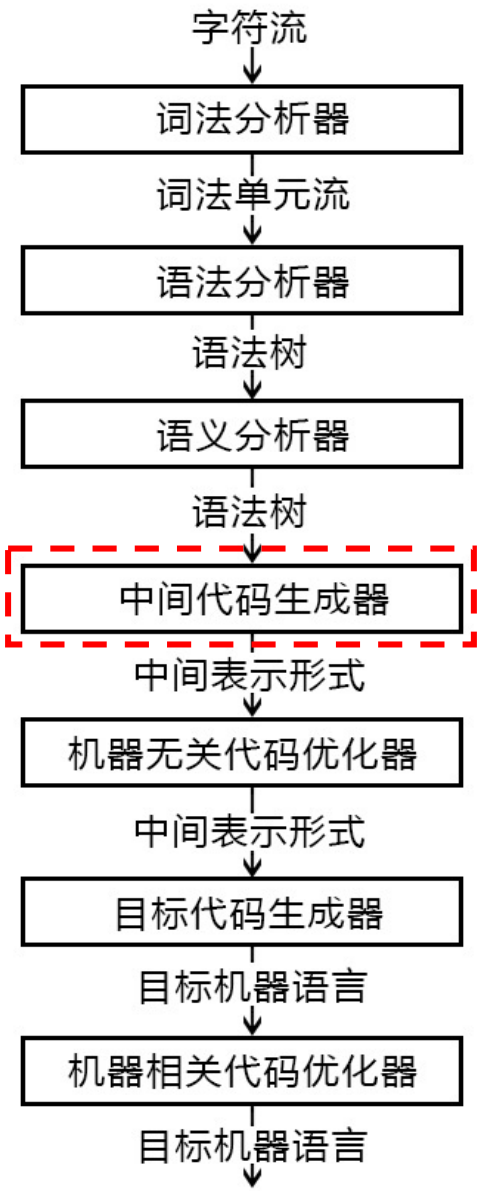
TAIL

符号表是用于存放标识符的属性信息的数据结构

语义分析的主要任务

- 收集标识符的属性信息
- 语义检查
  - 变量（包括数组、指针、结构体）或过程未经声明就使用
  - 变量（包括数组、指针、结构体）或过程名重复声明
  - 运算分量类型不匹配
  - 操作符与操作数之间的类型不匹配
    - 赋值号左边出现一个只有右值的表达式
    - 数组下标不是整数
    - 对非数组变量使用数组访问操作符
    - 对非结构体类型变量使用“.”操作符
    - 对非过程名使用过程调用操作符
    - 过程调用的参数类型或数目不匹配
    - 函数返回类型有误

编译器的结构


字符流
词法分析器
词法单元流
语法分析器
语法树
语义分析器
语法树
中间代码生成器
中间表示形式
机器无关代码优化器
中间表示形式
目标代码生成器
目标机器语言
机器相关代码优化器
目标机器语言

常用的中间表示形式

- 三地址码 (Three-address Code)
  - 三地址码由类似于汇编语言的指令序列组成，每个指令最多有三个操作数(operand)
- 语法结构树/语法树 (Syntax Trees)

常用的三地址指令

序号	指令类型	指令形式
1	赋值指令	$x = y$ op z $x =$ op y
2	复制指令	$x = y$
3	条件跳转	if x relop y goto n
4	非条件跳转	goto n
5	参数传递	param x
6	过程调用 函数调用	call p, n $y =$call p, n
7	过程返回	return x
8	数组引用	$y = x[i]$
9	数组赋值	$x[i] = y$
10	地址及 指针操作	$x =\& y$ $x =* y$ $*x = y$

地址可以具有如下形式之一

- 源程序中的名字 (*name*)
- 常量 (*constant*)
- 编译器生成的临时变量(*temporary*)

三地址指令的表示

- 四元式 (Quadruples)
  - $(op, arg_1, arg_2, result)$
- 三元式 (Triples)
  - (op, arg_1, agr_2)
- 间接三元式 (Indirect triples)

三地址指令的四元式表示

- $x = y$ op z (op , y , z , x)
- $x =$ op y (op , y , _ , x)
- $x = y$ (= , y , _ , x)
- if x relop y goto n (relop , x , y , n)
- goto n (goto , _ , _ , n)
- param x (param, _ , _ , x)

  $y =$ call p, n (call , p , n , y)

  return x (return, _ , _ , x)
- $y = x[i]$ (=[] , x , i , y)

  $x[i] = y$ ([]= , y , x , i)
- $x = \&y$ (& , y , _ , x)

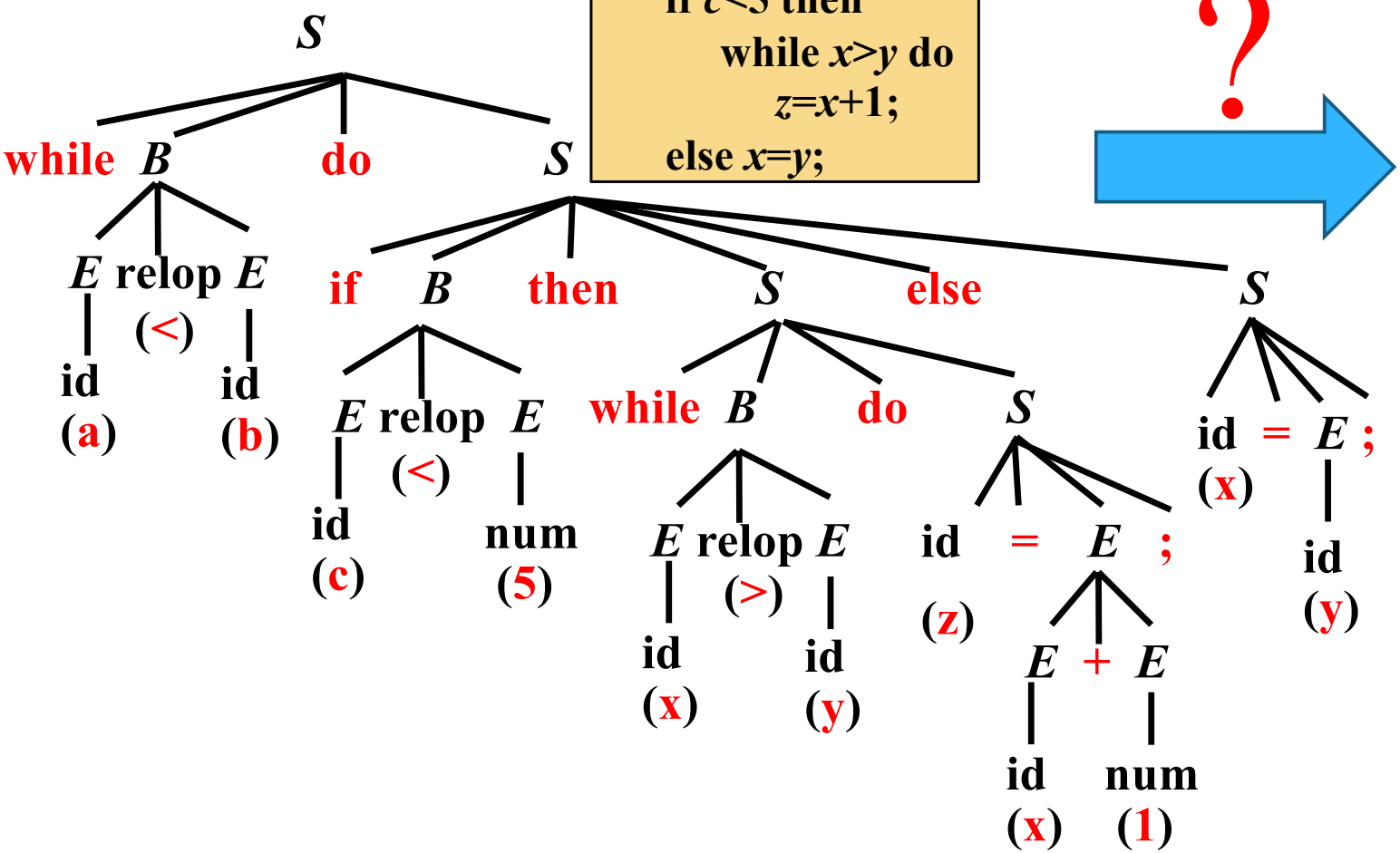
  $x = *y$ (=* , y , _ , x)

  $*x = y$ (*= , y , _ , x)

三地址指令序列唯一确定了运算完成的顺序

中间代码生成的例子

```
while a<b do
    if c<5 then
        while x>y do
            z=x+1;
    else x=y;
```

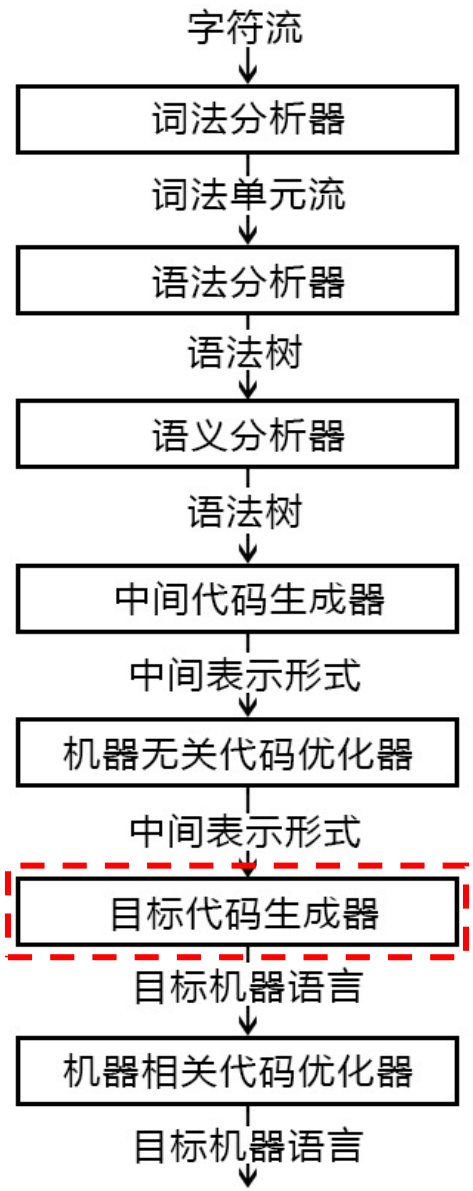


?

100: (j<, a ,b , 102)
101: (j , - , - , 112)
102: (j<, c , 5 , 104)
103: (j , - , - , 110)
104: (j>, x , y , 106)
105: (j , - , - , 100)
106: (+ , x , 1 , t_1)
107: (= , t_1 , - , z)
108: (j , - , - , 104)
109: (j , - , - , 100)
110: (= , y , - , x)
111: (j , - , - , 100)
112:

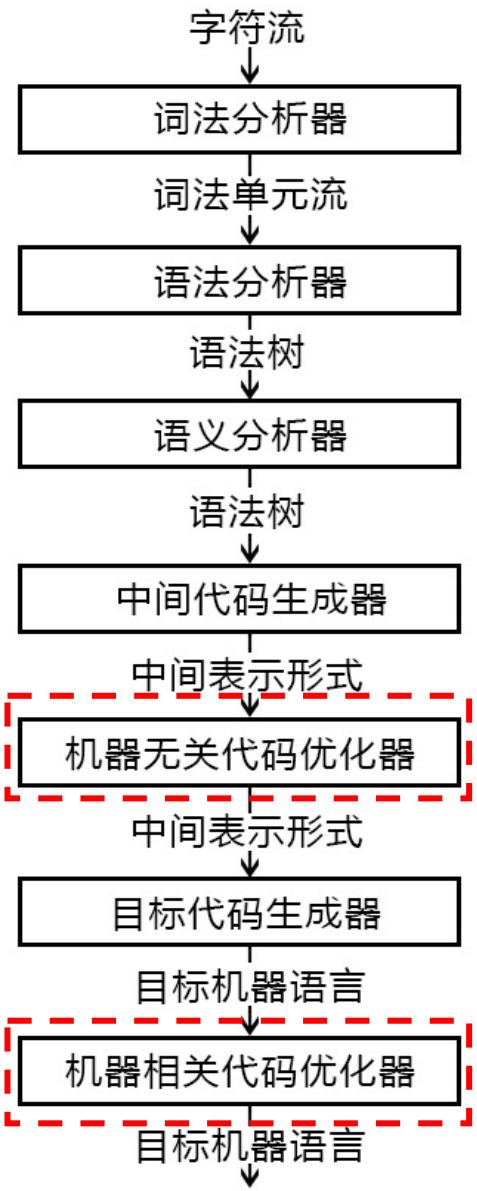
编译器的结构

- 目标代码生成以源程序的中间表示形式作为输入，并把它映射到目标语言
- 目标代码生成的一个重要任务是为程序中使用的变量合理分配寄存器



编译器的结构

- 代码优化
  - 为改进代码所进行的等价程序变换，使其运行得更快一些、占用空间更少一些，或者二者兼顾



提纲

1.1 什么是编译

1.2 编译系统的结构

1.3 编译程序的生成

1.4 为什么要学习编译原理

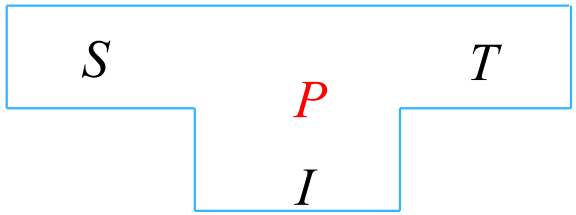
1.5 编译技术的应用

编译程序的生成

- 1970年以前，几乎所有的编译程序都是用机器语言编写的
  - 优点：更好地发挥硬件系统的效率
  - 缺点：可读性、可靠性、可维护性、编制效率差
- 1980年以后，通常用高级语言来编写编译程序（自展技术）

编译器的T形图

- *P*：编译器（程序）
  - *I*：实现语言
  - *S*：输入的源语言程序
  - *T*：输出的可执行的目标程序



注意：

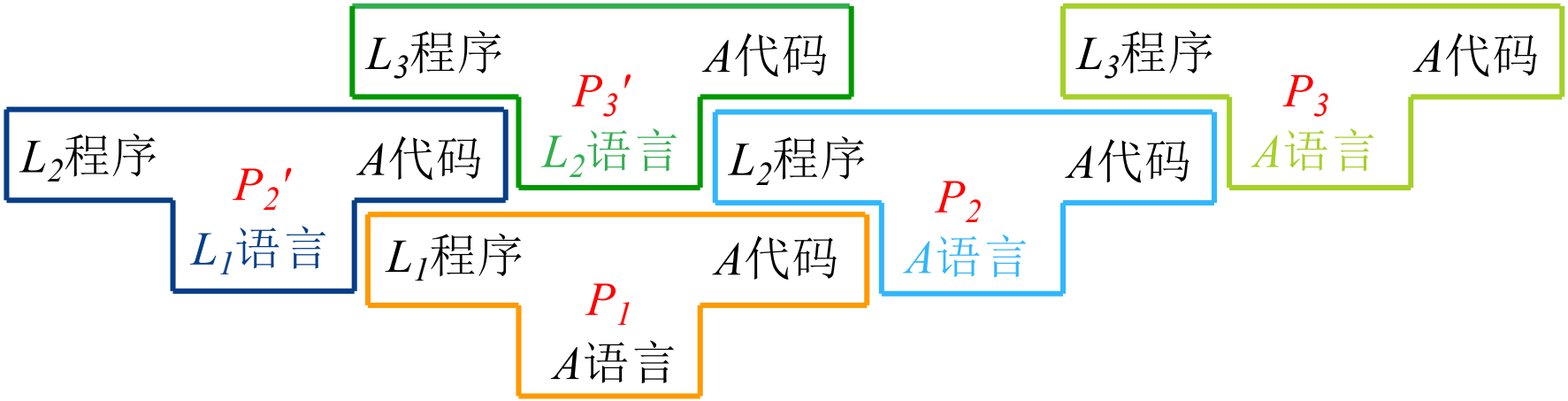
（1）*I*表示的是语言，而*S*和*T*表示的是程序

（2）*T*形图的上端体现了编译器的功能，即从哪种语言到哪种语言的翻译

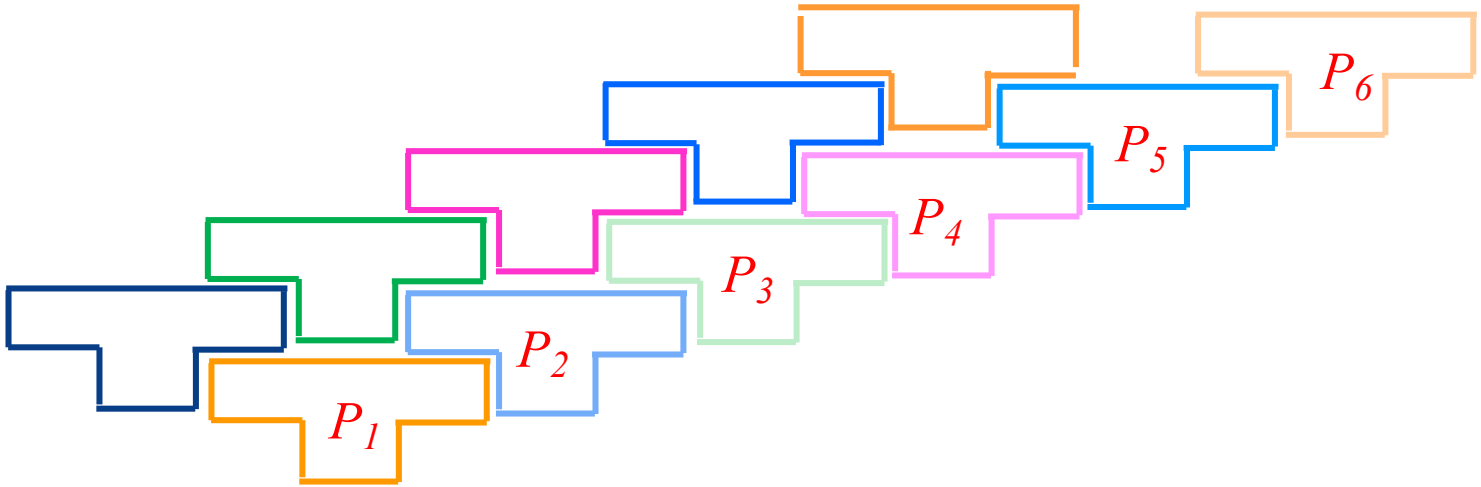
（3）*T*对应于某机器语言

自展（在同一台机器上实现不同语言的编译器）

- 给定P_1：A机器上运行的高级语言L_1的编译器
- 构造P_2：A机器上运行的高级语言L_2的编译器
- 构造P_3：A机器上运行的高级语言L_3的编译器



自展（在同一台机器上实现不同语言的编译器）

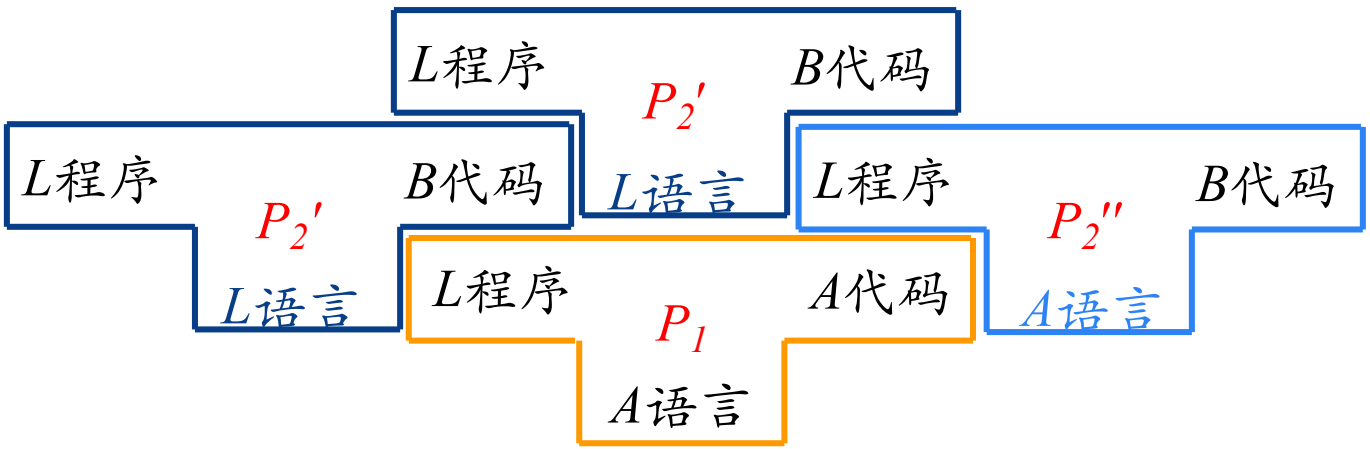


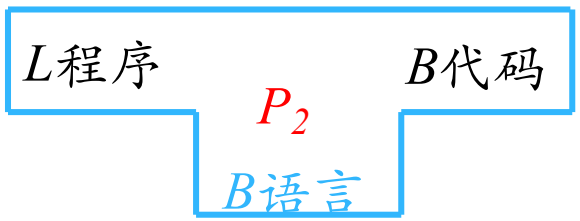
编译程序的生成

- 1970年以前，几乎所有的编译程序都是用机器语言编写的
  - 优点：更好地发挥硬件系统的效率
  - 缺点：可读性、可靠性、可维护性、编制效率差
- 1980年以后，通常用高级语言来编写编译程序（自展技术）
- 编译器的移植
  - 有时也称为交叉编译，是指将一台机器上运行的编译器进行处理，构造出在另一台机器上可以运行的编译器

编译器的移植

- 给定P_1： A机器上运行的高级语言L的编译器
- 构造P_2： B机器上运行的高级语言L的编译器





编译程序的生成

- 1970年以前，几乎所有的编译程序都是用机器语言编写的
  - 优点：更好地发挥硬件系统的效率
  - 缺点：可读性、可靠性、可维护性、编制效率差
- 1980年以后，通常用高级语言来编写编译程序（自展技术）
- 编译器的移植
- 编译器的自动生成
  - LEX： 词法分析程序生成器
  - YACC：语法分析程序生成器

提纲

1.1 什么是编译

1.2 编译系统的结构

1.3 编译程序的生成

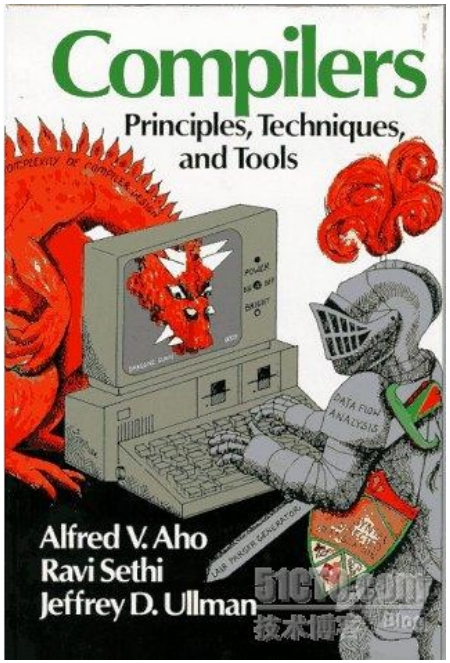
1.4 为什么要学习编译原理

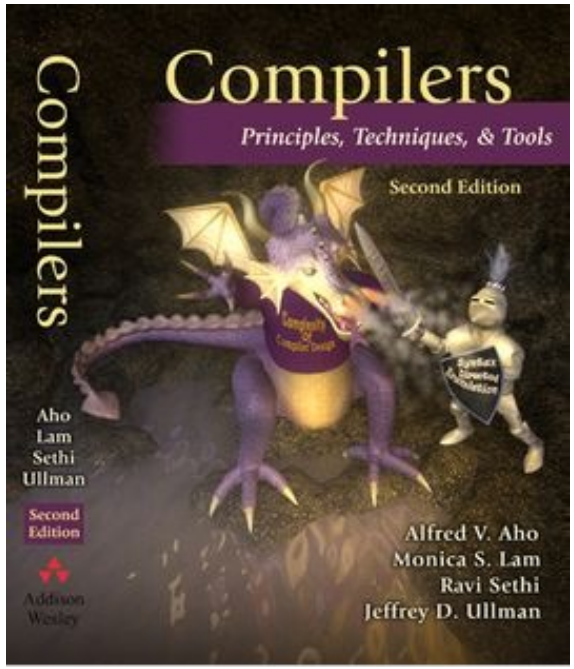
1.5 编译技术的应用

1.4 为什么要学习编译原理

编写编译器的原理和技术具有十分普遍的意义，以至于在每个计算机科学家的研究生涯中，本课程中的原理和技术都会反复用到。

——Alfred V.Aho





通过本课程的学习

- 更深刻地理解高级语言程序的内部运行机制
- 教给我们如何严谨地去思考、编写程序
- 编译原理涉及了计算机科学求解问题的基本思路和方法，即问题的“形式化描述→自动化处理”
- 所涉及的理论和方法在很多领域都会被用到
  - 自然语言处理、模式识别、人工智能、……
- 很多应用软件都会用到编译技术

提纲

1.1 什么是编译

1.2 编译系统的结构

1.3 编译程序的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

1.5 编译技术的应用

- 结构化编辑器（Structure editors）
  - 引导用户在语言的语法约束下编制程序
  - 能自动地提供关键字和与其匹配的关键字

1.5 编译技术的应用

- 结构化编辑器（Structure editors）
- 智能打印机（Pretty printers）
  - 对程序进行分析，打印出结构清晰的程序
    - 注释以一种特殊的字体打印
    - 根据各个语句在程序的层次结构中的嵌套深度进行缩进

1.5 编译技术的应用

- 结构化编辑器（Structure editors）
- 智能打印机（Pretty printers）
- 静态检测器（Static checkers）
  - 静态定位程序中的错误
    - 释放空指针或已释放过的指针
    - 检测出程序中永远不能被执行的语句

1.5 编译技术的应用

- 结构化编辑器（Structure editors）
- 智能打印机（Pretty printers）
- 静态检测器（Static checkers）
- 文本格式器（Text formatters）
  - 文本格式器处理的字符流中除了需要排版输出的字符以外，还包含一些用来说明字符流中的段落、图表或者上标和下标等数学结构的命令

1.5 编译技术的应用

- 结构化编辑器（Structure editors）
- 智能打印机（Pretty printers）
- 静态检测器（Static checkers）
- 文本格式器（Text formatters）
- 数据库查询解释器（ Database Query Interpreters ）
  - 数据库查询语句由包含了关系和布尔运算的谓词组成。查询解释器把这些谓词翻译成数据库命令，在数据库中查询满足条件的记录。

1.5 编译技术的应用

- 结构化编辑器（Structure editors）
- 智能打印机（Pretty printers）
- 静态检测器（Static checkers）
- 文本格式器（Text formatters）
- 数据库查询解释器（ Database Query Interpreters ）
- 高级语言的翻译工具

本章小结

什么是编译

编译系统的结构

编译程序的生成

为什么要学习编译原理

编译技术的应用

课程主要内容

- 1. 绪论 (2学时)
- 2. 语言及其文法 (2学时)
- 3. 词法分析 (2学时)
- 4. 语法分析 (10学时)
- 5. 语法制导翻译 (6学时)
- 6. 中间代码生成 (8学时)
- 7. 运行时的存贮组织 (3学时)
- 8. 代码优化 (5学时)
- 9. 代码生成 (2学时)

结束